



Ray Casting

Lecture 2 Fall 2025

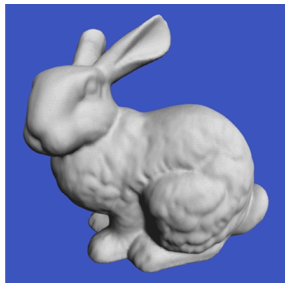
Press Space for next page →

Announcements

- Assignment 1 is due this week (Friday, Sep 12, 11:59 PM)
 - Office hours for A1 online this Friday
- Assignment Grace Tokens:
 - 10 tokens that each provide an additional 12 hours to submit penalty free.
 - Read more on the course Github.

Last Time: Photography

Images captured from the real world.



Object



Light



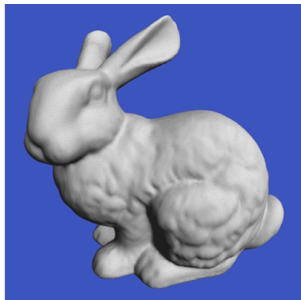
Camera



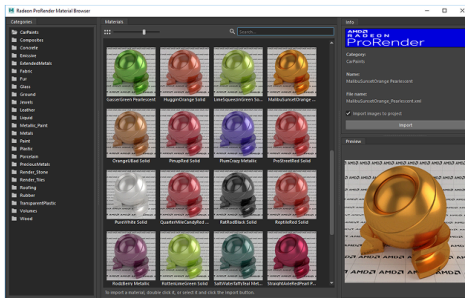
Image

Rendering: From Real to Virtual

How do we generate images using digital scenes?



Object



Materials



Image

Overview of Rendering

What problem are we actually solving?

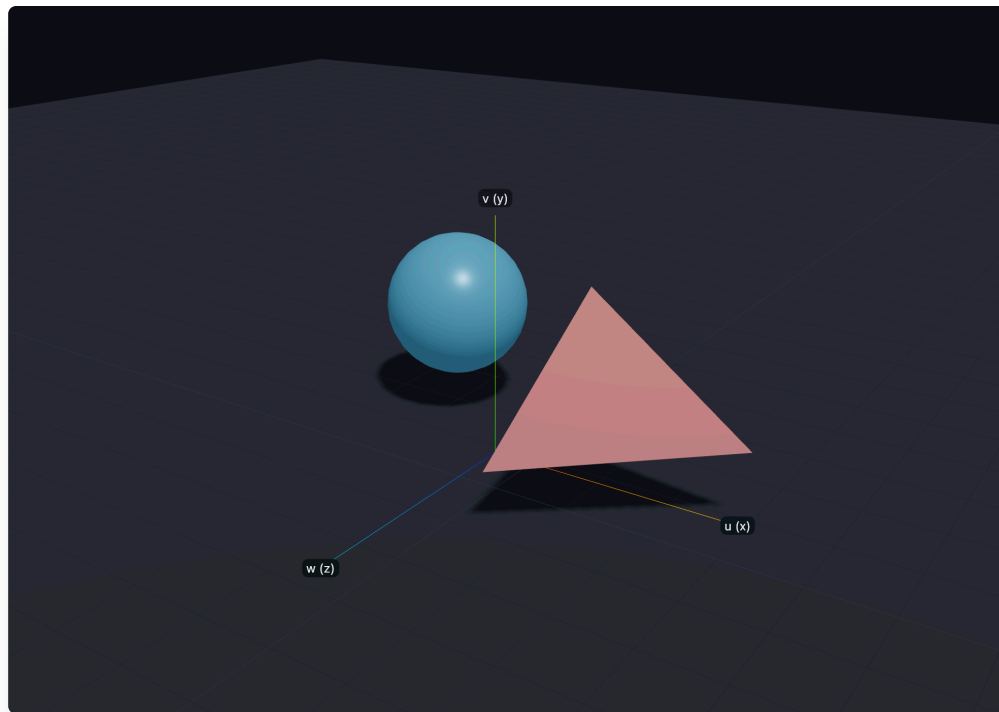
Defining the image function $I(x, y)$

What problem are we actually solving?

Defining the image function $I(x, y)$

3D Scene

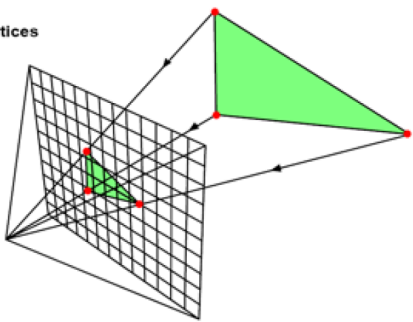
A simple virtual scene with basic geometry



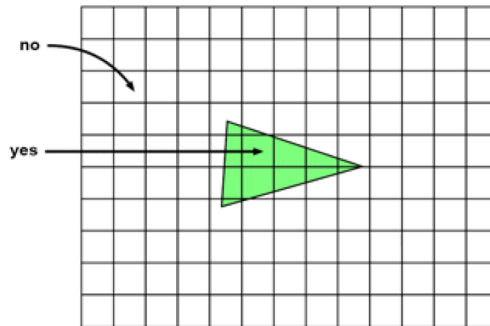
Rasterization

Projecting to the image plane

1) Project vertices

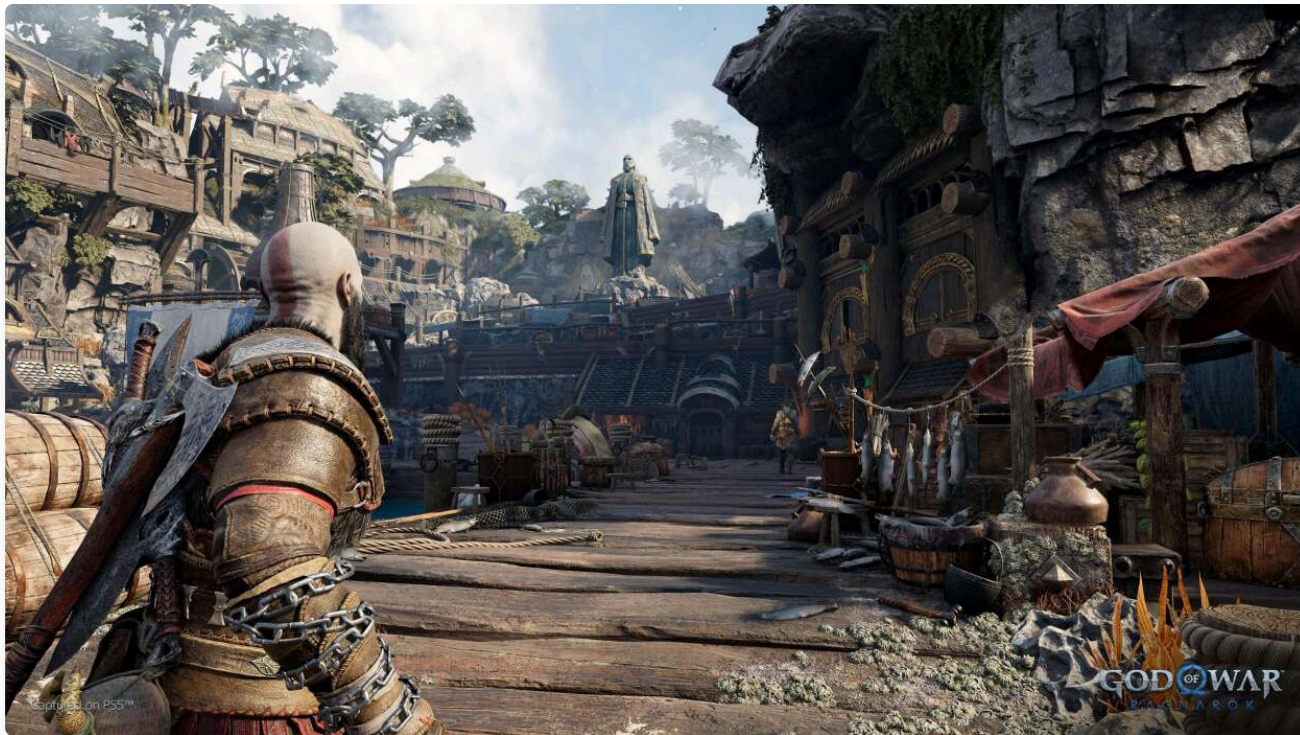


© www.scratchapixel.com



Rasterization

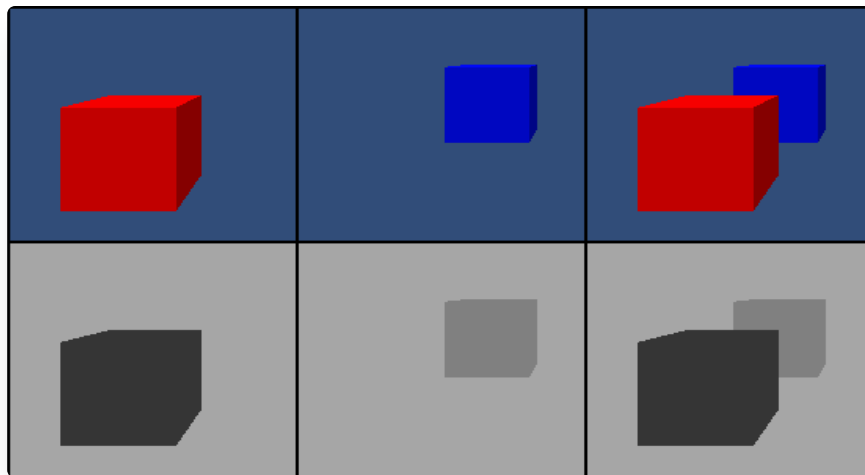
Projecting to the image plane



Rasterization Loop

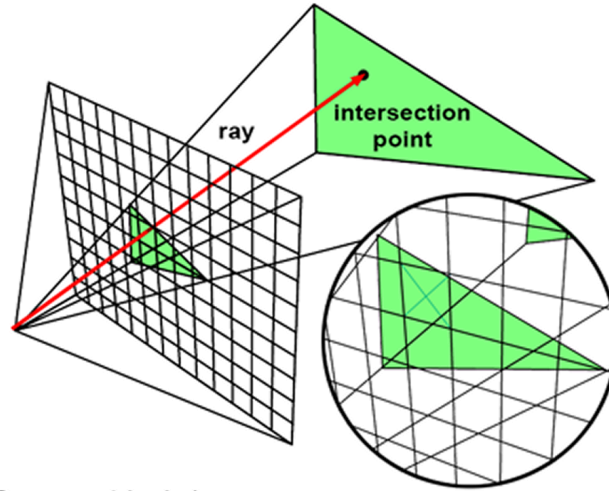
Object-order Rendering: Objects first, then pixels.

```
1  for each scene object {  
2    for each pixel {  
3      if (object affects pixel) {  
4        Do something  
5      }  
6    }  
7  }
```



Ray Casting

Projecting rays from pixels into the scene



© www.scratchapixel.com

Ray Casting

Projecting rays from pixels into the scene



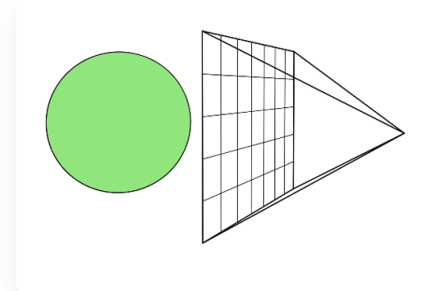
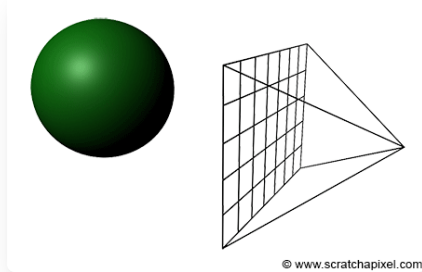
<https://www.scratchapixel.com/lessons/3d-basic-rendering/rasterization-practical-implementation/overview-rasterization-algorithm.html>

<https://computergraphics.stackexchange.com/questions/4585/mirror-reflections-ray-tracing-or-rasterisation>

Ray Casting Loop

Image-order rendering: Pixels first, then objects.

```
1  for each pixel {  
2    generate a ray  
3    for each object {  
4      if (ray intersects object) {  
5        Do something  
6      }  
7    }  
8  }
```



Difference in Loops

Trade-offs

```
for each pixel {  
    generate a ray  
    for each object {  
        if (ray intersects object) {  
            Do something  
        }  
    }  
}
```

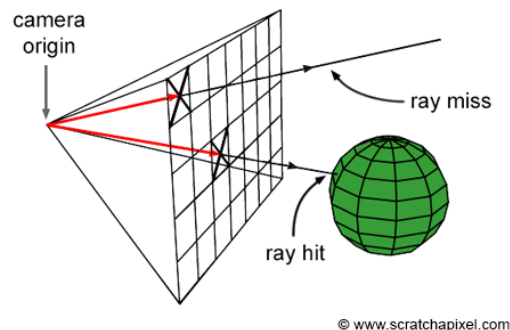
- Faster and Parallel
- Good for realistic lighting
- Works with many geometric representations

```
1  for each scene object {  
2      for each pixel {  
3          if (object affects pixel) {  
4              Do something  
5          }  
6      }  
7  }
```

- Excellent for realistic lighting
- Works with many geometric representations (meshes, implicit surfaces, volumes, voxels)
- Slow and noisy

Ray Casting Overview

```
1  for each pixel {  
2    generate a ray  
3    for each object {  
4      if (ray intersects object) {  
5        Do something  
6      }  
7    }  
8  }
```



- ☐ What am I shooting?
- ☐ Where is being shot?
- ☐ How do I know if I hit something?

Ray

Origin + Direction

Ray Practice

Practice

Question: If I have a point $\vec{a} = (1, 1)$ and a point $\vec{b} = (3, 2)$, define a ray that starts at point $\vec{a}$ and goes through point $\vec{b}$.

Ray Practice

Practice

Question: If I have a point $\vec{a} = (1, 1)$ and a point $\vec{b} = (3, 2)$, define a ray that starts at point $\vec{a}$ and goes through point $\vec{b}$.

Answer: $\vec{p}(t) = \vec{a} + t(\vec{b} - \vec{a}) = (1, 1) + t((3, 2) - (1, 1)) = (1, 1) + t(2, 1)$

Ray Practice

Practice

Question: If I have a point $\vec{a} = (1, 1)$ and a point $\vec{b} = (3, 2)$, define a ray that starts at point $\vec{a}$ and goes through point $\vec{b}$.

Answer: $\vec{p}(t) = \vec{a} + t(\vec{b} - \vec{a}) = (1, 1) + t((3, 2) - (1, 1)) = (1, 1) + t(2, 1)$

Question: Now we have a point $\vec{c} = (19, 10)$. What value of t will get me to point $\vec{c}$ using the ray you defined?

Ray Practice

Practice

Question: If I have a point $\vec{a} = (1, 1)$ and a point $\vec{b} = (3, 2)$, define a ray that starts at point $\vec{a}$ and goes through point $\vec{b}$.

Answer: $\vec{p}(t) = \vec{a} + t(\vec{b} - \vec{a}) = (1, 1) + t((3, 2) - (1, 1)) = (1, 1) + t(2, 1)$

Question: Now we have a point $\vec{c} = (19, 10)$. What value of t will get me to point $\vec{c}$ using the ray you defined?

$$(19, 10) = (1, 1) + t(2, 1)$$

$$(19, 10) = (1 + 2t, 1 + t)$$

$$t = 9$$

Ray Practice

Practice

Question: If I have a point $\vec{a} = (1, 1)$ and a point $\vec{b} = (3, 2)$, define a ray that starts at point $\vec{a}$ and goes through point $\vec{b}$.

Answer: $\vec{p}(t) = \vec{a} + t(\vec{b} - \vec{a}) = (1, 1) + t((3, 2) - (1, 1)) = (1, 1) + t(2, 1)$

Question: Now we have a point $\vec{c} = (19, 10)$. What value of t will get me to point $\vec{c}$ using the ray you defined?

$$(19, 10) = (1, 1) + t(2, 1)$$

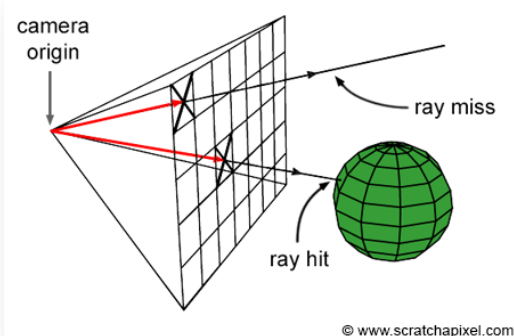
$$(19, 10) = (1 + 2t, 1 + t)$$

$$t = 9$$

***Important:** If we have 2 points, we have a way to define a ray.

Ray Casting Mechanisms

```
1  for each pixel {  
2    generate a Ray  
3    for each object {  
4      if (ray intersects object) {  
5        Do something  
6      }  
7    }  
8  }
```



- ☒ What am I shooting? A Ray: $p(t) = e + t \cdot d$
- ☐ Where is being shot?
- ☐ How do I know if I hit something?

3D scene <-- ? --> 2D pixel grid

What ties the 3D scene and the 2D pixel grid?

The 3D image plane is a conceptual plane that sits between the 3D scene and the 2D pixel grid.

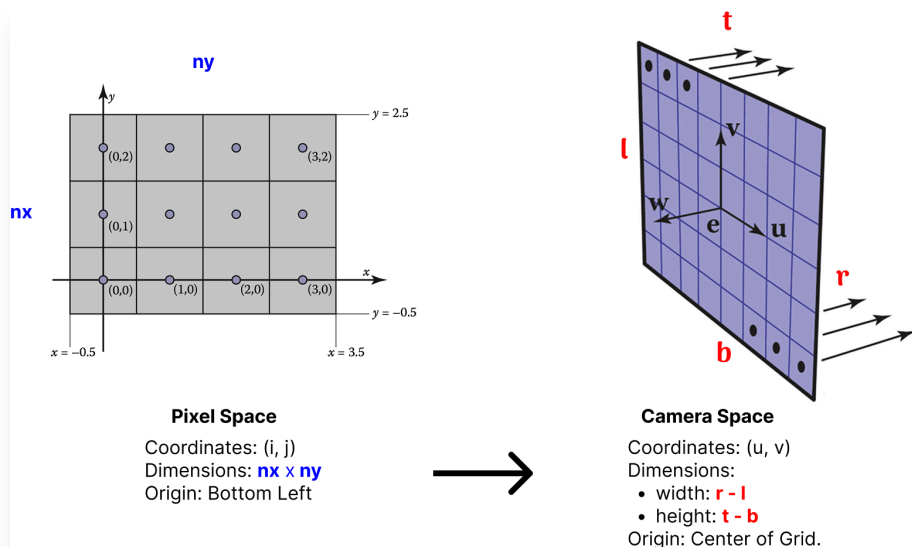
3D scene <-- ? --> 2D pixel grid

What ties the 3D scene and the 2D pixel grid?

The 3D image plane is a conceptual plane that sits between the 3D scene and the 2D pixel grid.

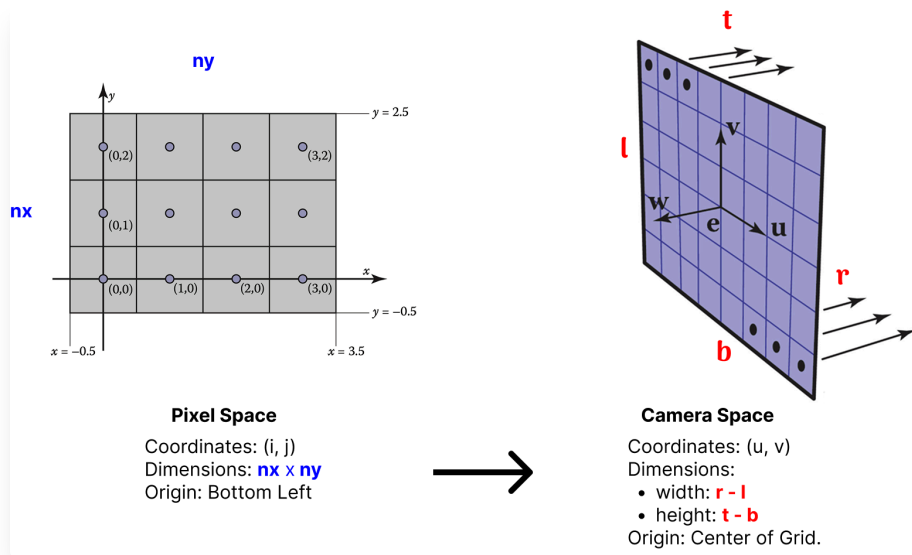
Mapping 2D pixel grid to 3D image plane

How does each pixel (i, j) map to a point (u, v) on the image plane?



Mapping 2D pixel grid to 3D image plane

How does each pixel (i, j) map to a point (u, v) on the image plane?

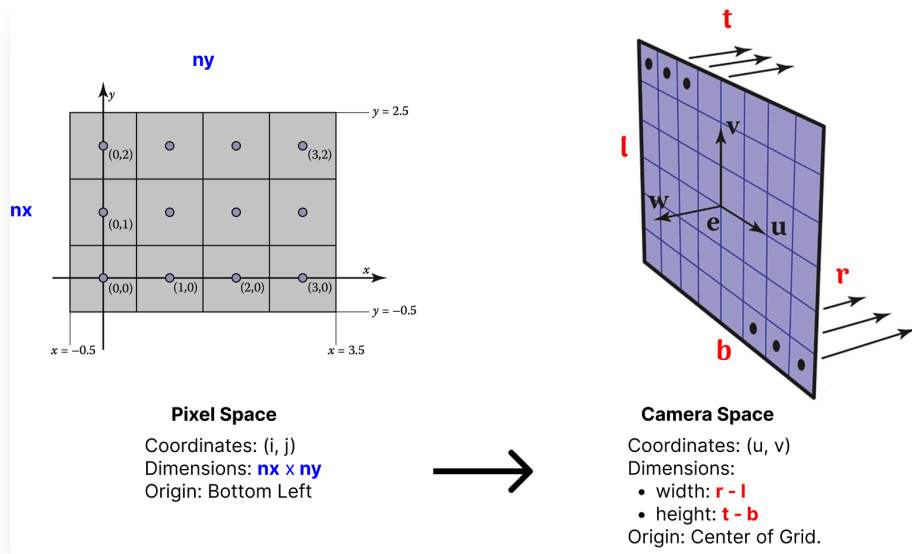


Q: What is the bottom-left corner of the pixel grid? Of the Image Plane?

Q: What is the top-right corner of the pixel grid? Of the Image Plane?

Mapping 2D pixel grid to 3D image plane

How does each pixel (i, j) map to a point (u, v) on the image plane?

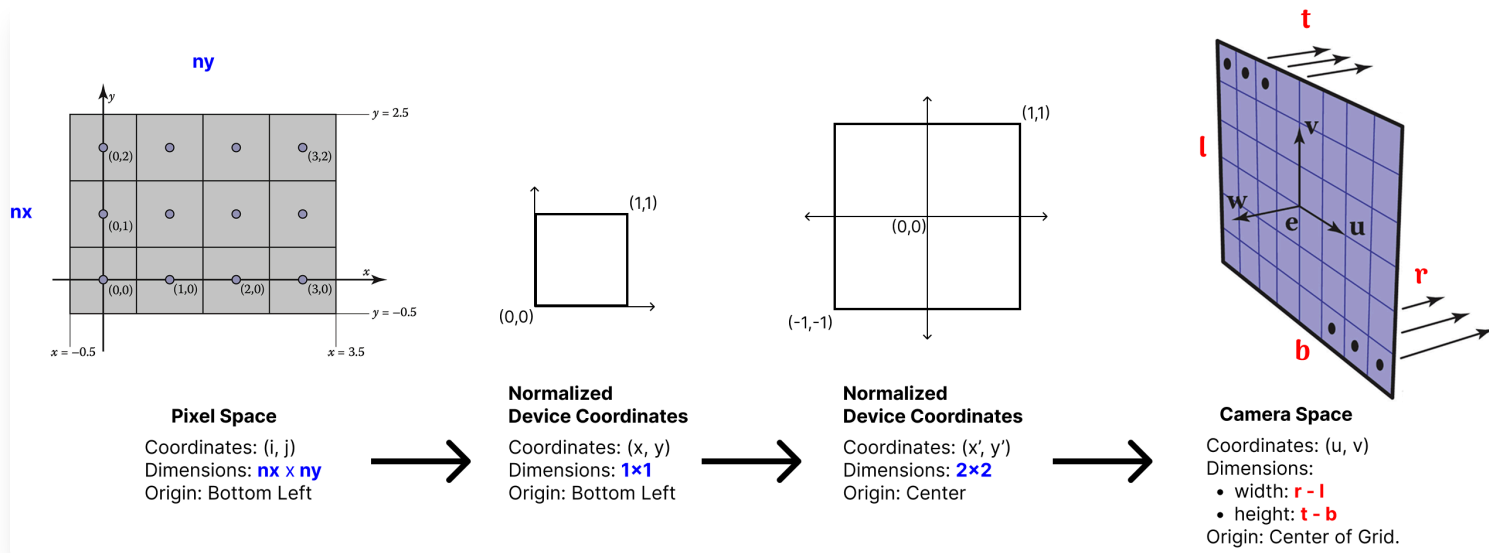


Q: What is the bottom-left corner of the pixel grid? Of the Image Plane? $(-0.5, -0.5)$ and (l, b)

Q: What is the top-right corner of the pixel grid? Of the Image Plane? $(nx - 0.5, ny - 0.5)$ and (r, t)

Mapping 2D pixel grid to 3D image plane

How does each pixel (i, j) map to a point (u, v) on the image plane?

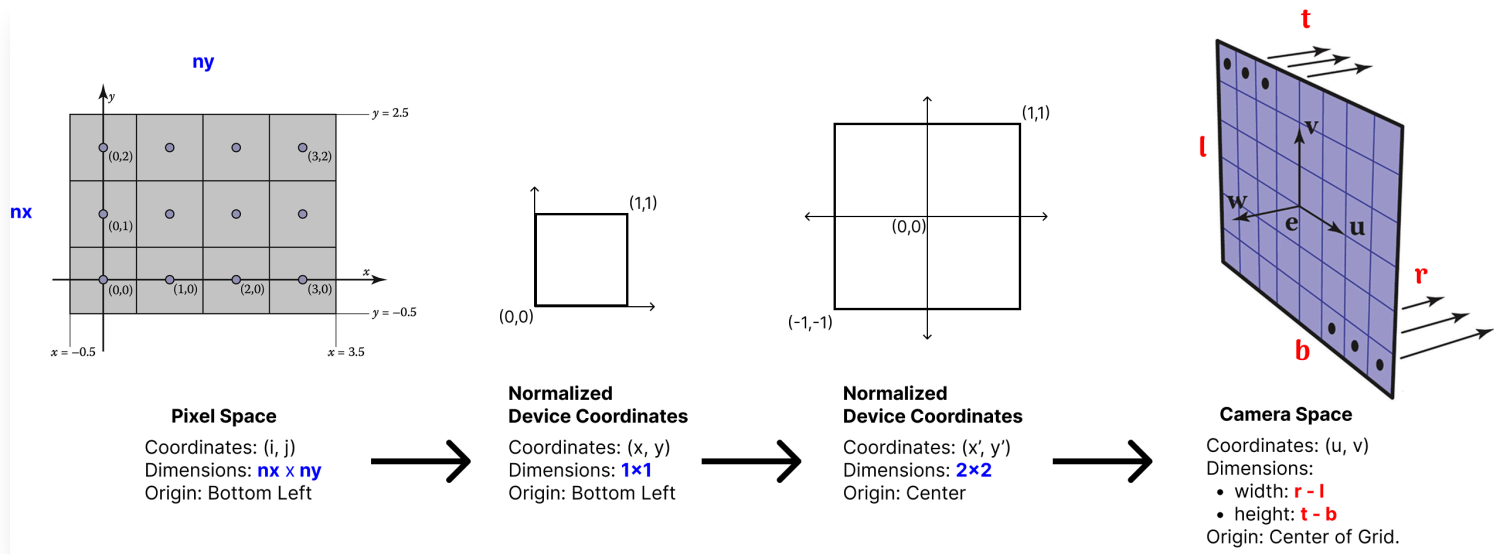


Q: What is the bottom-left corner of the pixel grid? Of the Image Plane? $(-0.5, -0.5)$ and (l, b)

Q: What is the top-right corner of the pixel grid? Of the Image Plane? $(nx - 0.5, ny - 0.5)$ and (r, t)

Mapping 2D pixel grid to 3D image plane

How does each pixel (i, j) map to a point (u, v) on the image plane?



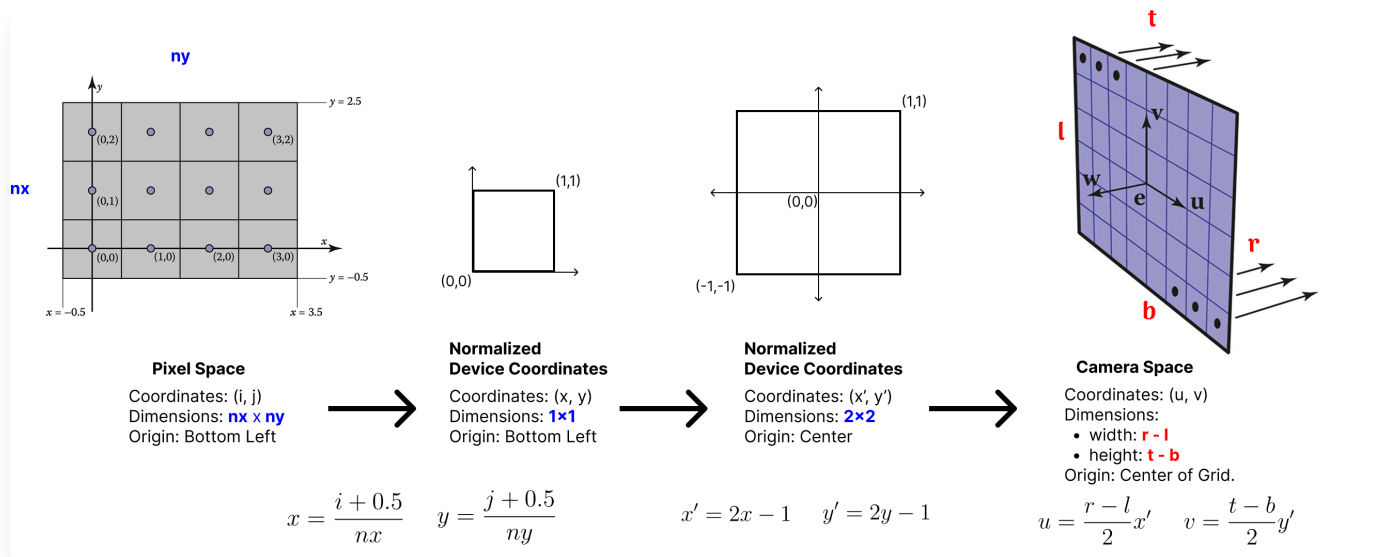
Q: What is the bottom-left corner of the pixel grid? Of the Image Plane? $(-0.5, -0.5)$ and (l, b)

Q: What is the top-right corner of the pixel grid? Of the Image Plane? $(nx - 0.5, ny - 0.5)$ and (r, t)

Try to derive the mapping from (i, j) to (u, v) .

Mapping 2D pixel grid to 3D image plane

How does each pixel (i, j) map to a point (u, v) on the image plane?



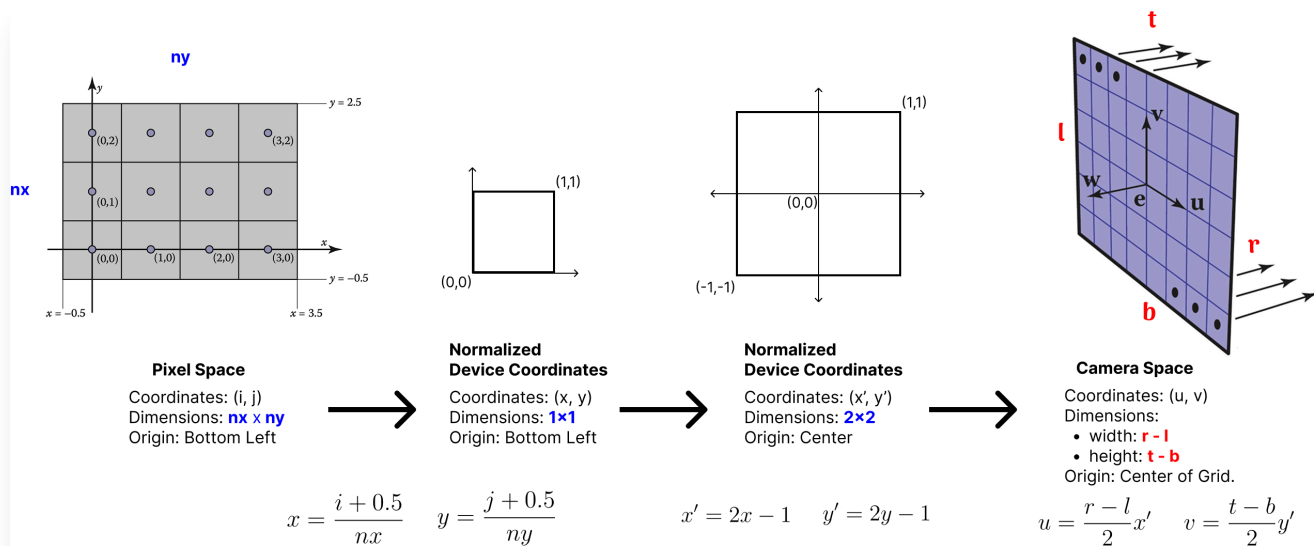
Q: What is the bottom-left corner of the pixel grid? Of the Image Plane? $(-0.5, -0.5)$ and (l, b)

Q: What is the top-right corner of the pixel grid? Of the Image Plane? $(nx - 0.5, ny - 0.5)$ and (r, t)

Try to derive the mapping from (i, j) to (u, v) .

Mapping 2D pixel grid to 3D image plane

How does each pixel (i, j) map to a point (u, v) on the image plane?

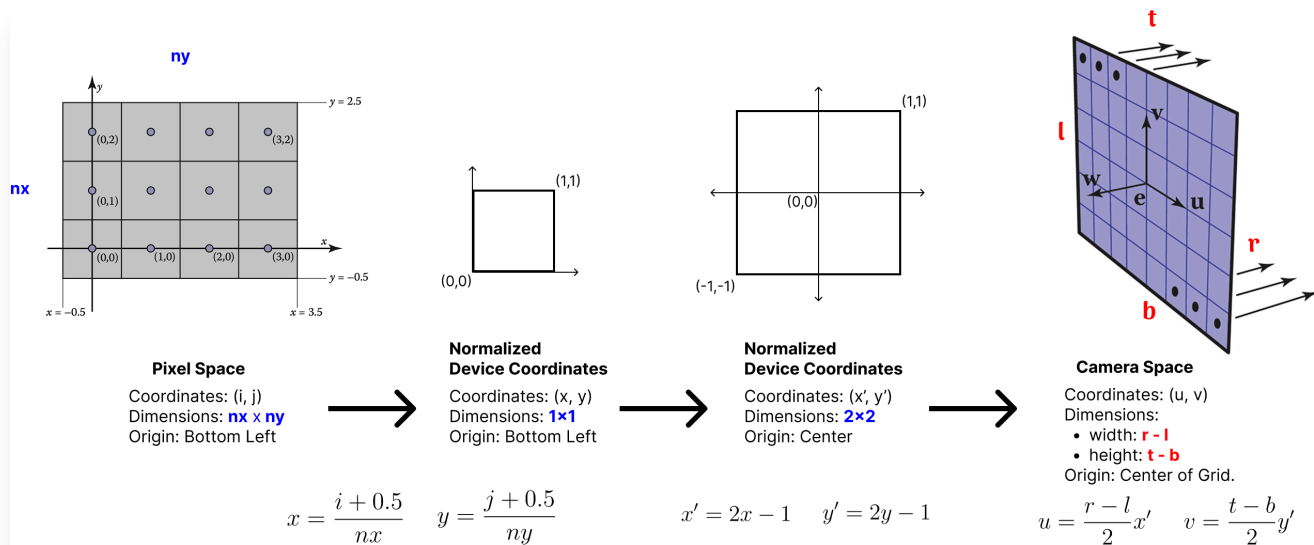


$$u = \frac{(r - l)}{nx} (i + 0.5) - \frac{(r - l)}{2}$$

$$v = \frac{(t - b)}{ny} (j + 0.5) - \frac{(t - b)}{2}$$

Mapping 2D pixel grid to 3D image plane

How does each pixel (i, j) map to a point (u, v) on the image plane?

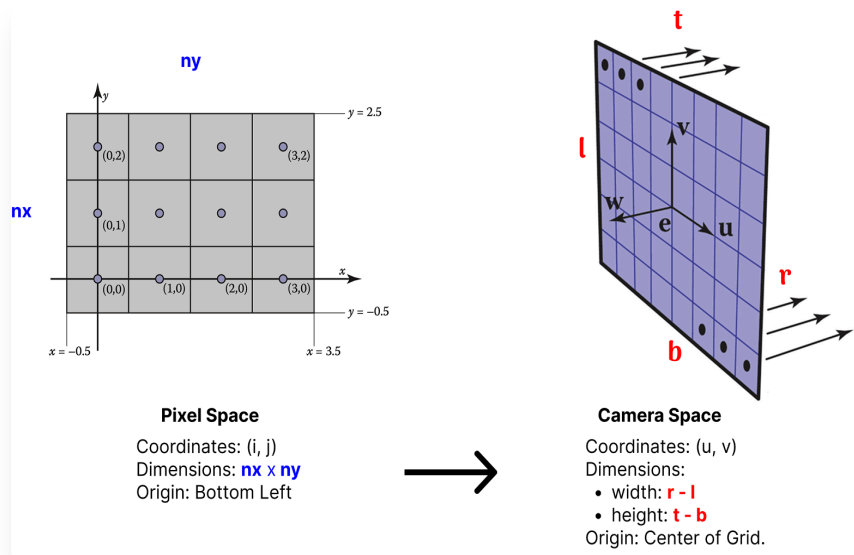


$$u = \frac{width}{nx} (i + 0.5) - \frac{width}{2}$$

$$v = \frac{height}{ny} (j + 0.5) - \frac{height}{2}$$

Shooting rays

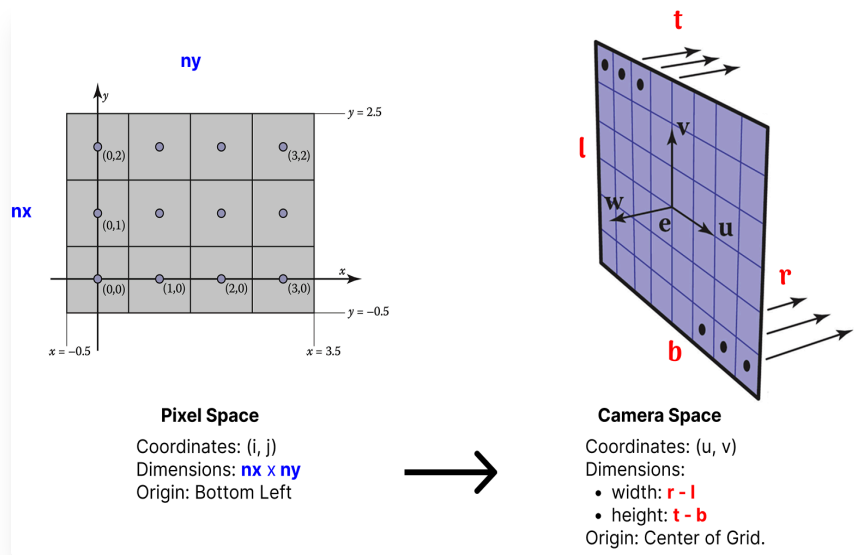
Which direction do we shoot rays from the image plane?



$$u = \frac{width}{nx}(i + 0.5) - \frac{width}{2}$$
$$v = \frac{height}{ny}(j + 0.5) - \frac{height}{2}$$

Shooting rays

Which direction do we shoot rays from the image plane?

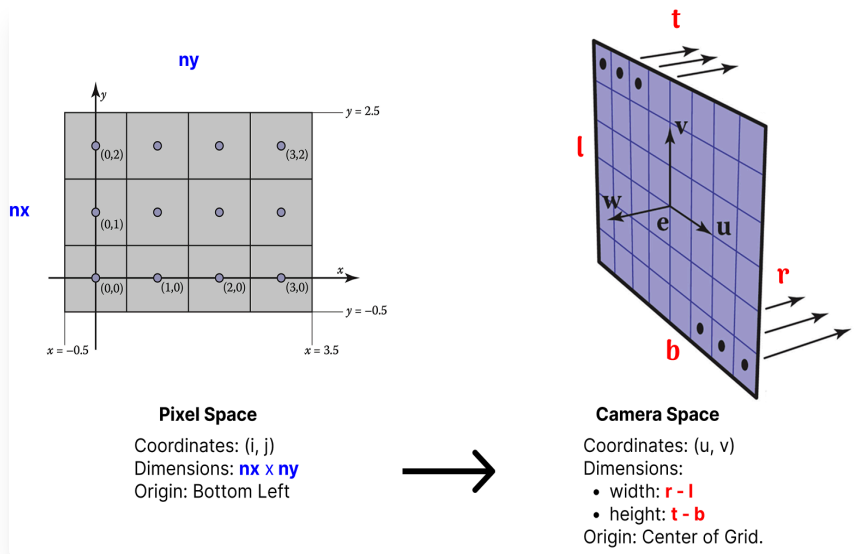


$$\vec{p}(t) = \vec{e} + t\vec{d}$$

$$u = \frac{width}{nx}(i + 0.5) - \frac{width}{2}$$
$$v = \frac{height}{ny}(j + 0.5) - \frac{height}{2}$$

Shooting rays

Which direction do we shoot rays from the image plane?



$$\vec{p}(t) = \vec{e} + t\vec{d}$$

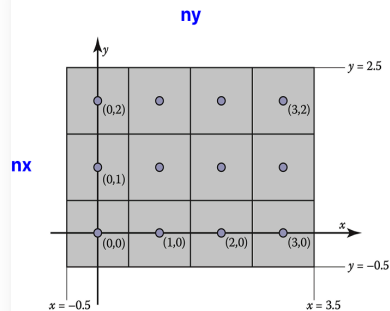
$$\vec{e} = (u, v, 0)$$

$$\vec{d} = -\vec{w}$$

$$u = \frac{width}{nx}(i + 0.5) - \frac{width}{2}$$
$$v = \frac{height}{ny}(j + 0.5) - \frac{height}{2}$$

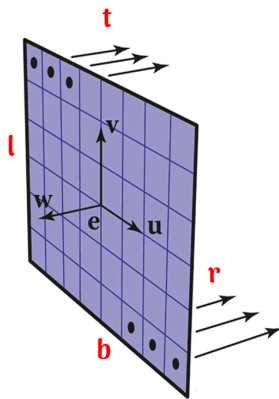
Orthographic Projection

Same direction, different origin.



Pixel Space

Coordinates: (i, j)
Dimensions: $nx \times ny$
Origin: Bottom Left



Camera Space

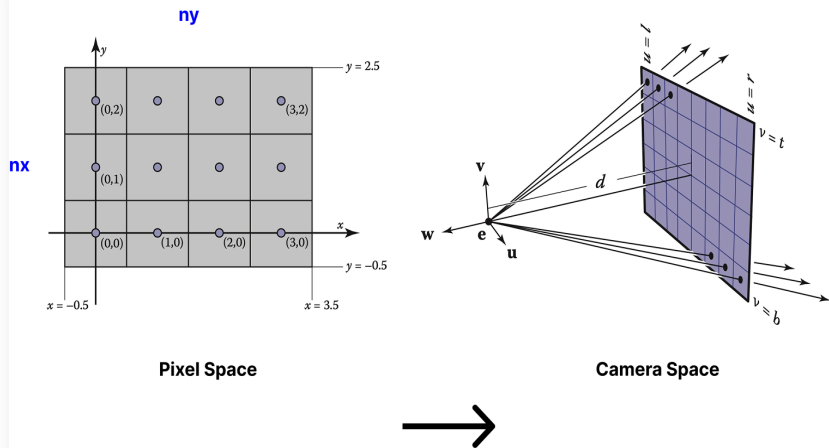
Coordinates: (u, v)
Dimensions:
• width: $r - l$
• height: $t - b$
Origin: Center of Grid.

```
for each pixel {  
    u = width/nx * (i + 0.5) - width/2  
    v = height/ny * (j + 0.5) - height/2  
    e = (u, v, 0)  
    d = -w  
  
    for each object {  
        if (ray intersects object) {  
            Do something  
        }  
    }  
}
```

Demo

Perspective Projection

Same origin, different direction.



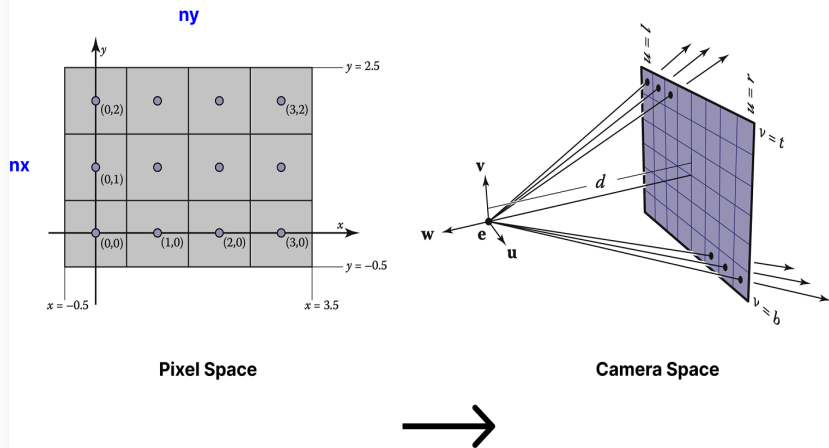
$$\vec{p}(t) = \vec{e} + t\vec{d}$$

$$\vec{e} = ?$$

$$\vec{d} = ?$$

Perspective Projection

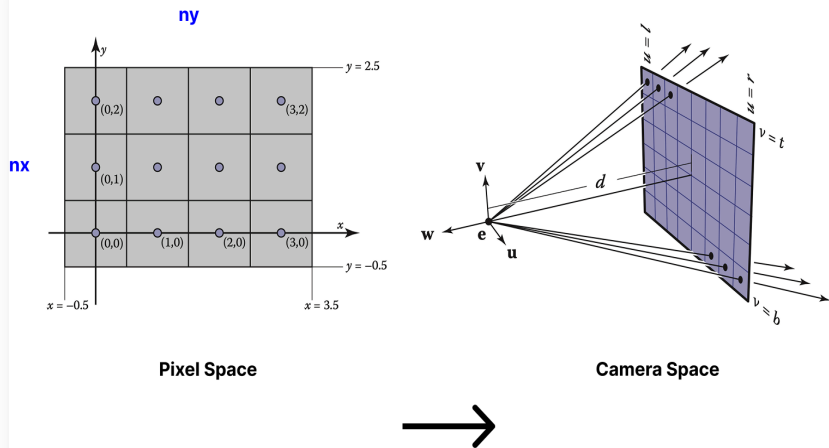
Same origin, different direction.



$$\vec{p}(t) = \vec{e} + t\vec{d}$$
$$\vec{e} = (0, 0, 0)$$
$$\vec{d} = ?$$

Perspective Projection

Same origin, different direction.



$$\vec{p}(t) = \vec{e} + t\vec{d}$$

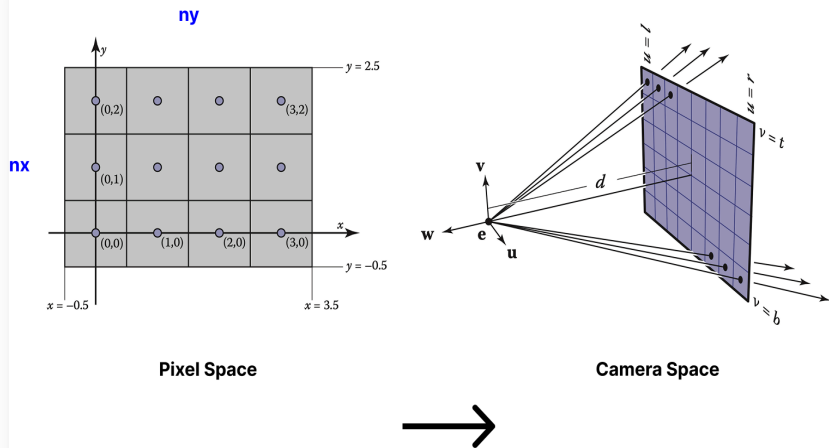
$$\vec{e} = (0, 0, 0)$$

$$\vec{d} = (u, v, d)$$

$$u = \frac{width}{nx}(i + 0.5) - \frac{width}{2}$$
$$v = \frac{height}{ny}(j + 0.5) - \frac{height}{2}$$

Perspective Projection

Same origin, different direction.



```
1  for each pixel {  
2    e = (0, 0, 0)  
3    u = width/nx * (i + 0.5) - width/2  
4    v = height/ny * (j + 0.5) - height/2  
5    d = (u, v, focal_length)  
6  
7    for each object {  
8      if (ray intersects object) {  
9        Do something  
10     }  
11   }  
12 }
```

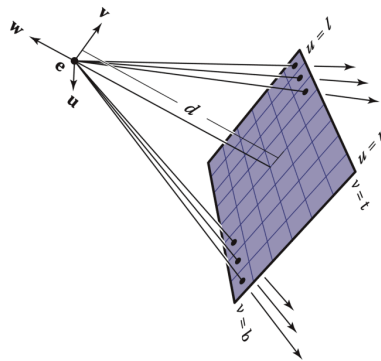
Demo

Ray Equation in Camera Space

General form:

$$\vec{p}(t) = \vec{e} + t(u(i)\vec{u} + v(j)\vec{v} - d\vec{w})$$

$$\vec{p}(t) = \vec{e} + t \begin{bmatrix} \vec{u} & \vec{v} & \vec{w} \end{bmatrix} \begin{bmatrix} u(i) \\ v(j) \\ -d \end{bmatrix}$$



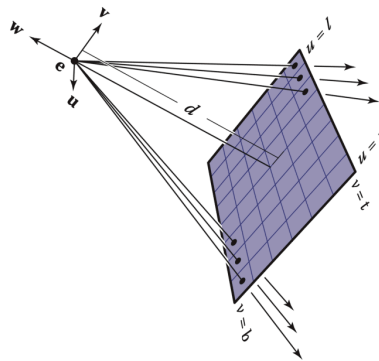
Ray Equation in Camera Space

General form:

$$\vec{p}(t) = \vec{e} + t(u(i)\vec{u} + v(j)\vec{v} - d\vec{w})$$

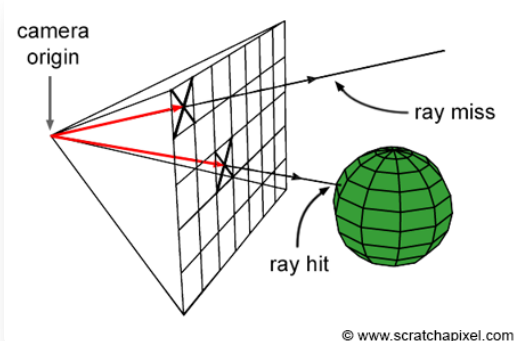
$$\vec{p}(t) = \vec{e} + t \begin{bmatrix} \vec{u} & \vec{v} & \vec{w} \end{bmatrix} \begin{bmatrix} u(i) \\ v(j) \\ -d \end{bmatrix}$$

$\begin{bmatrix} \vec{u} & \vec{v} & \vec{w} \end{bmatrix}$ Is a camera matrix



Ray Casting Mechanisms

```
1  for each pixel {  
2    generate a Ray  
3    for each object {  
4      if (ray intersects object) {  
5        Do something  
6      }  
7    }  
8  }
```



- ☒ ~~x~~ What am I shooting? A Ray: $p(t) = e + t \cdot d$
- ☒ ~~x~~ Where is the ray being shot? From the camera towards the image plane.
- ☐ How do I know if I hit something?

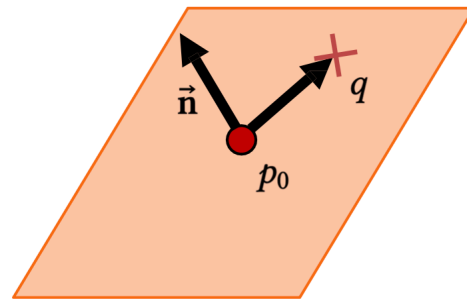
Intersection Test

- Plane
- Sphere
- Triangle

Plane

Intersection with a plane.

$$\vec{n} \cdot (\vec{q} - \vec{p}_0) = 0$$

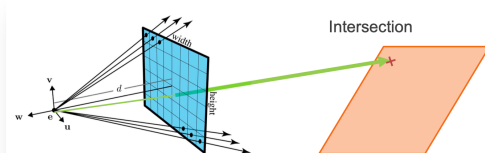


Plane

Intersection with a plane.

$$\vec{n} \cdot (\vec{q} - \vec{p}_0) = 0$$

$$\vec{p}(t) = \vec{e} + t\vec{d}$$



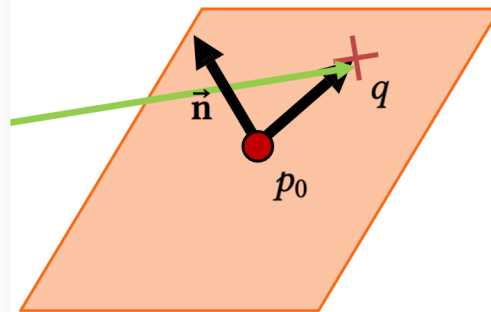
Plane

Intersection with a plane.

$$\vec{n} \cdot (\vec{q} - \vec{p}_0) = 0$$

$$\vec{p}(t) = \vec{e} + t\vec{d}$$

$$\vec{n} \cdot ((\vec{e} + t\vec{d}) - \vec{p}_0) = 0$$



Plane

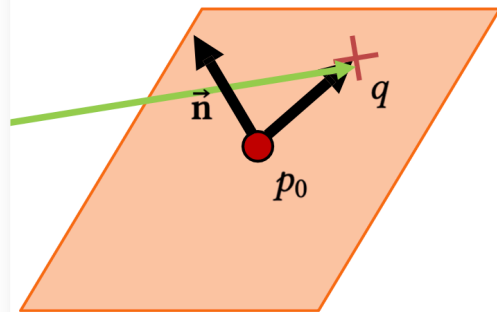
Intersection with a plane.

$$\vec{n} \cdot (\vec{q} - \vec{p}_0) = 0$$

$$\vec{p}(t) = \vec{e} + t\vec{d}$$

$$\vec{n} \cdot ((\vec{e} + t\vec{d}) - \vec{p}_0) = 0$$

$$\vec{n} \cdot (\vec{e} - \vec{p}_0) + t(\vec{n} \cdot \vec{d}) = 0$$



Plane

Intersection with a plane.

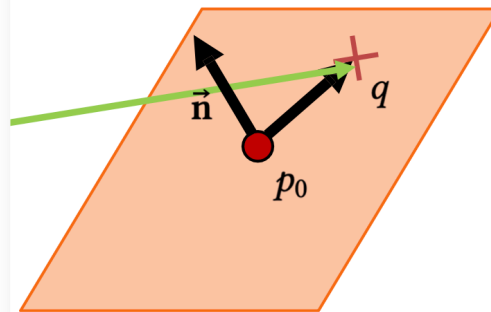
$$\vec{n} \cdot (\vec{q} - \vec{p}_0) = 0$$

$$\vec{p}(t) = \vec{e} + t\vec{d}$$

$$\vec{n} \cdot ((\vec{e} + t\vec{d}) - \vec{p}_0) = 0$$

$$\vec{n} \cdot (\vec{e} - \vec{p}_0) + t(\vec{n} \cdot \vec{d}) = 0$$

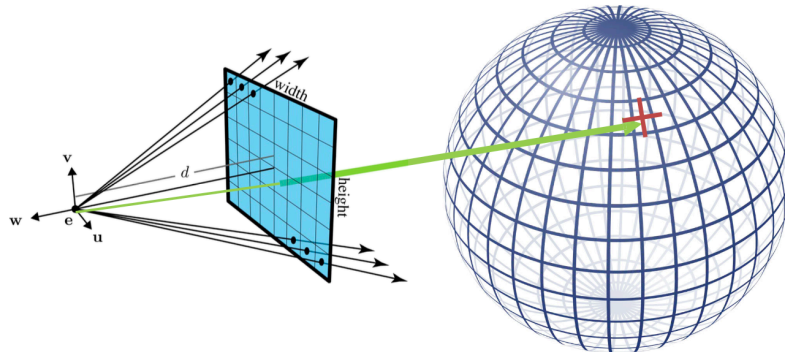
$$t = \frac{\vec{n} \cdot (\vec{p}_0 - \vec{e})}{\vec{n} \cdot \vec{d}}$$



Sphere

Intersection with a Sphere.

A sphere can be defined implicitly with a center $c = (x_c, y_c, z_c)$ and a radius r :

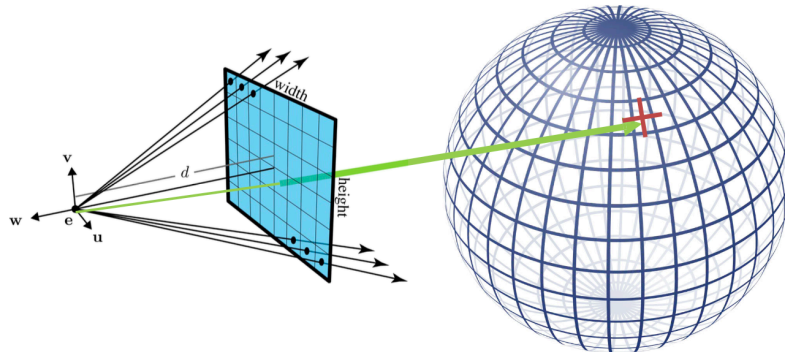


Sphere

Intersection with a Sphere.

A sphere can be defined implicitly with a center $c = (x_c, y_c, z_c)$ and a radius r :

$$(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - r^2 = 0$$



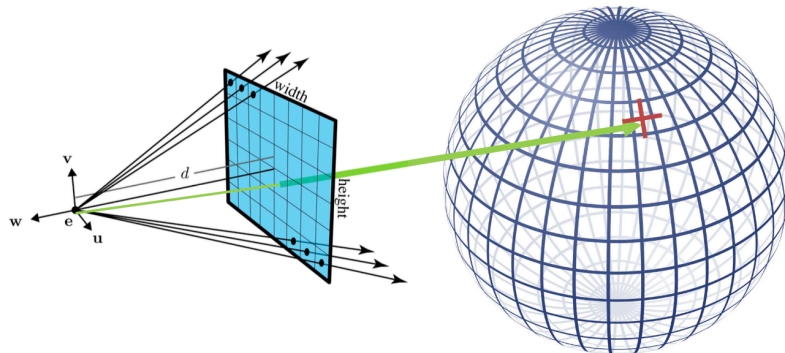
Sphere

Intersection with a Sphere.

A sphere can be defined implicitly with a center $\mathbf{c} = (x_c, y_c, z_c)$ and a radius r :

$$(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - r^2 = 0$$

$$(\vec{p} - \vec{c}) \cdot (\vec{p} - \vec{c}) - r^2 = 0$$



Sphere

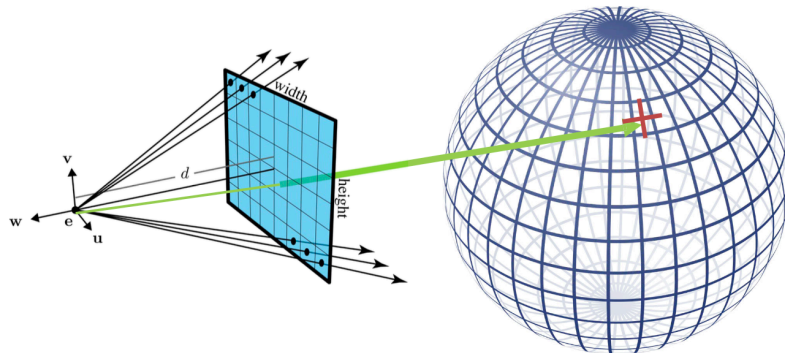
Intersection with a Sphere.

A sphere can be defined implicitly with a center $\mathbf{c} = (x_c, y_c, z_c)$ and a radius r :

$$(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - r^2 = 0$$

$$(\vec{p} - \vec{c}) \cdot (\vec{p} - \vec{c}) - r^2 = 0$$

$$(\vec{e} + t\vec{d} - \vec{c}) \cdot (\vec{e} + t\vec{d} - \vec{c}) - r^2 = 0$$



Sphere

Intersection with a Sphere.

A sphere can be defined implicitly with a center $c = (x_c, y_c, z_c)$ and a radius r :

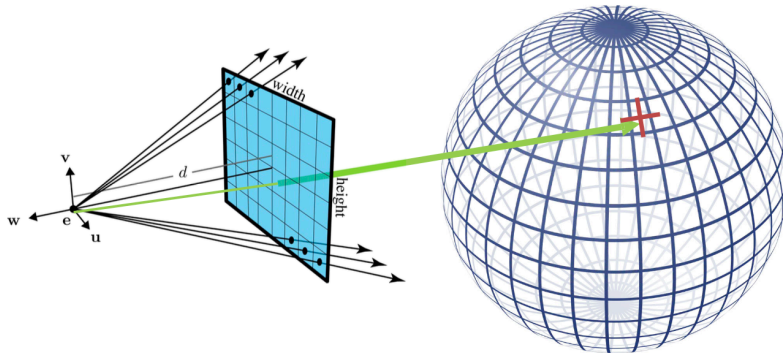
$$(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - r^2 = 0$$

$$(\vec{p} - \vec{c}) \cdot (\vec{p} - \vec{c}) - r^2 = 0$$

$$(\vec{e} + t\vec{d} - \vec{c}) \cdot (\vec{e} + t\vec{d} - \vec{c}) - r^2 = 0$$

Rearranging terms as:

$$(\vec{d} \cdot \vec{d})t^2 + 2\vec{d} \cdot (\vec{e} - \vec{c})t + (\vec{e} - \vec{c}) \cdot (\vec{e} - \vec{c}) - r^2 = 0$$



Sphere

Intersection with a Sphere.

A sphere can be defined implicitly with a center $c = (x_c, y_c, z_c)$ and a radius r :

$$(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - r^2 = 0$$

$$(\vec{p} - \vec{c}) \cdot (\vec{p} - \vec{c}) - r^2 = 0$$

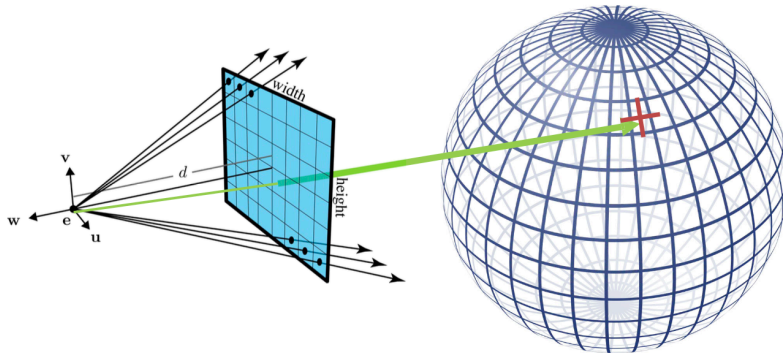
$$(\vec{e} + t\vec{d} - \vec{c}) \cdot (\vec{e} + t\vec{d} - \vec{c}) - r^2 = 0$$

Rearranging terms as:

$$(\vec{d} \cdot \vec{d})t^2 + 2\vec{d} \cdot (\vec{e} - \vec{c})t + (\vec{e} - \vec{c}) \cdot (\vec{e} - \vec{c}) - r^2 = 0$$

This looks familiar to us, it's a quadratic equation!

$$At^2 + Bt + C = 0$$



Sphere

Intersection with a Sphere.

A sphere can be defined implicitly with a center $c = (x_c, y_c, z_c)$ and a radius r :

$$(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - r^2 = 0$$

$$(\vec{p} - \vec{c}) \cdot (\vec{p} - \vec{c}) - r^2 = 0$$

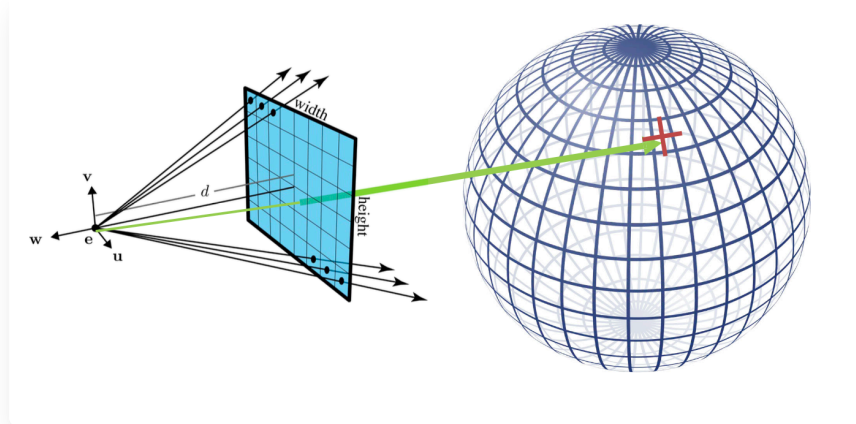
$$(\vec{e} + t\vec{d} - \vec{c}) \cdot (\vec{e} + t\vec{d} - \vec{c}) - r^2 = 0$$

Rearranging terms as:

$$(\vec{d} \cdot \vec{d})t^2 + 2\vec{d} \cdot (\vec{e} - \vec{c})t + (\vec{e} - \vec{c}) \cdot (\vec{e} - \vec{c}) - r^2 = 0$$

This looks familiar to us, it's a quadratic equation!

$$At^2 + Bt + C = 0$$

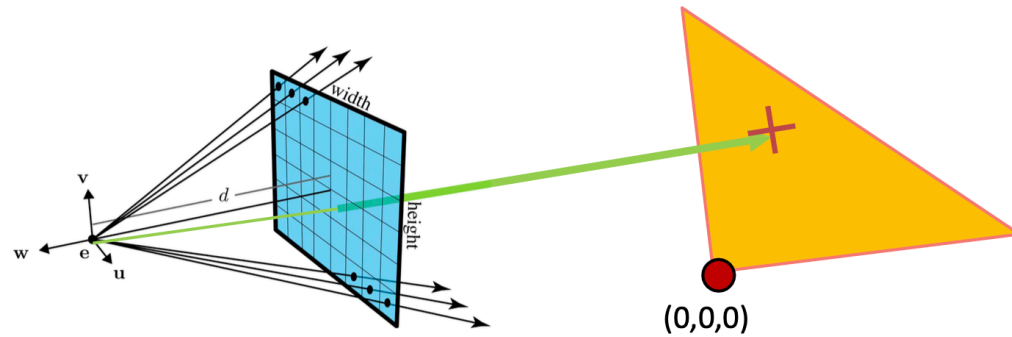


Looking at the discriminant, we can determine if we have an intersection:

- < 0 : no intersection
- $= 0$: one intersection (tangent)
- > 0 : two intersections

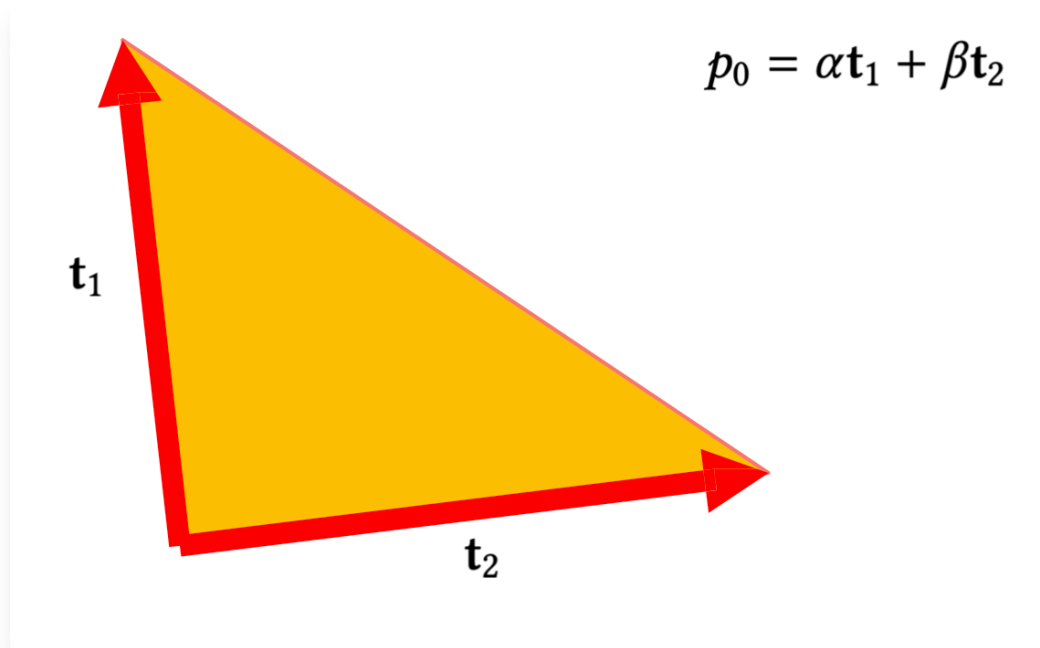
Triangle

Intersection with a Triangle.



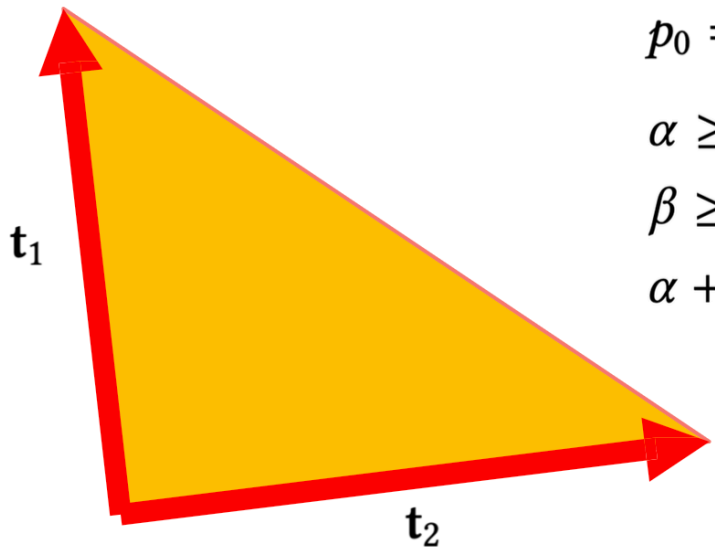
Triangle

Intersection with a Triangle.



Triangle

Intersection with a Triangle.



$$p_0 = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2$$

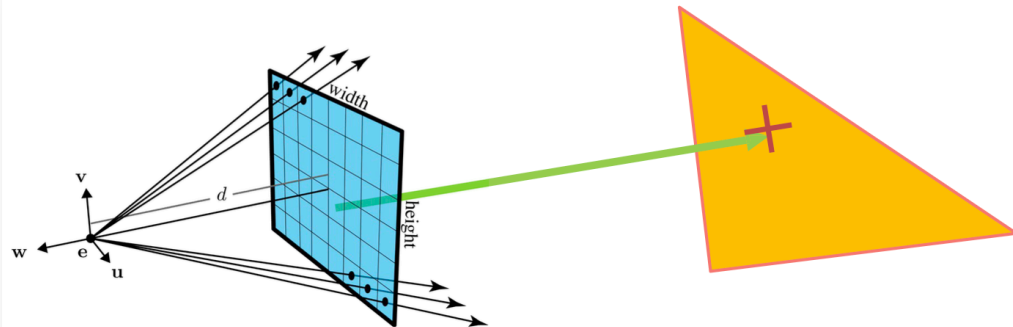
$$\alpha \geq 0$$

$$\beta \geq 0$$

$$\alpha + \beta \leq 1$$

Triangle

Intersection with a Triangle.



Triangle

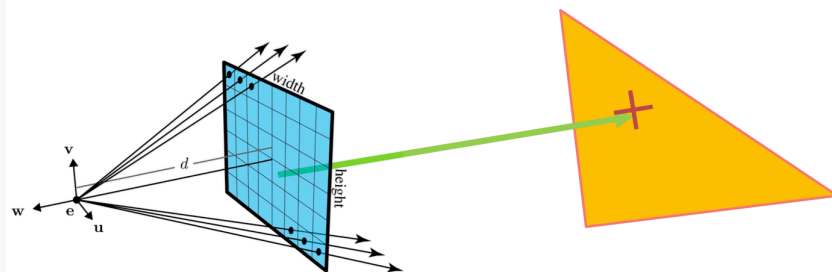
Intersection with a Triangle.

$$\mathbf{p}(t) = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2$$

$$\mathbf{e} + t\vec{\mathbf{d}} = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2$$

$$\mathbf{e} = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2 - t\vec{\mathbf{d}}$$

Check via equating point on surface with point on ray



Triangle

Intersection with a Triangle.

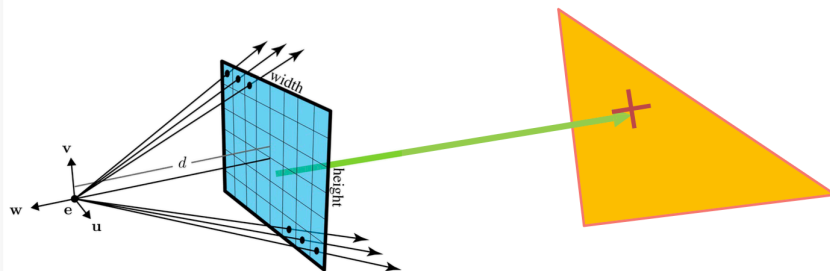
$$\mathbf{p}(t) = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2$$

$$\mathbf{e} + t\vec{\mathbf{d}} = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2$$

$$\mathbf{e} = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2 - t\vec{\mathbf{d}}$$

Check values of α and β

Check via equating point on surface with point on ray



Ray Casting Mechanisms

So what are some mechanisms we need to understand?

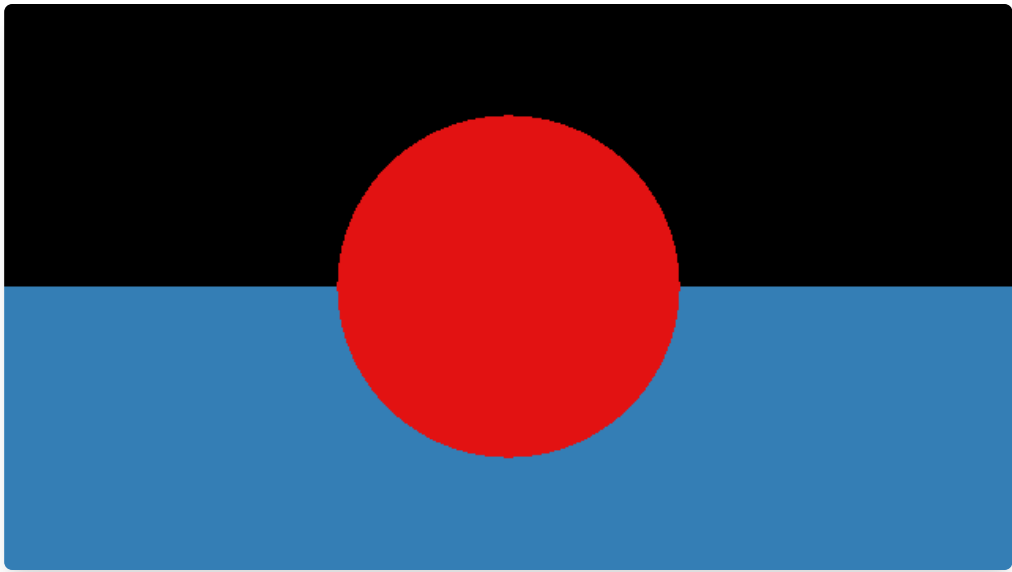
- ☒ ~~x~~ What am I shooting? A Ray: $p(t) = e + t \cdot d$
- ☒ ~~x~~ Where is the ray being shot? From the camera towards the image plane.
- ☒ ~~x~~ How do I know if I hit something? Intersection tests!

```
1  for each pixel {
2    ray = generateRay(i, j)
3    for each object {
4      if (intersects(ray, object)) {
5        setPixel(i, j, object.color)
6      }
7    }
8  }
```

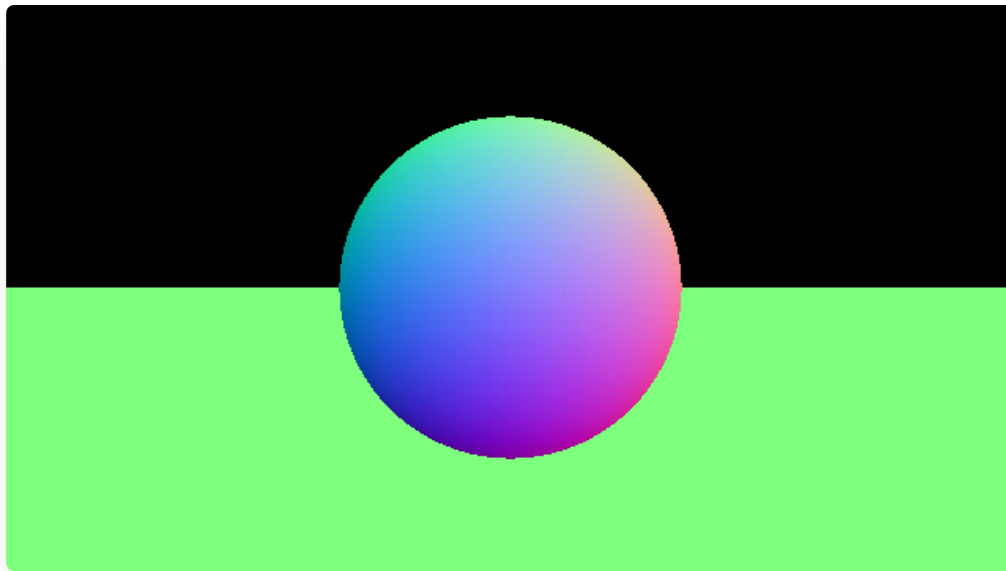
Output Types

- Object ID
- Surface Normal
- Depth

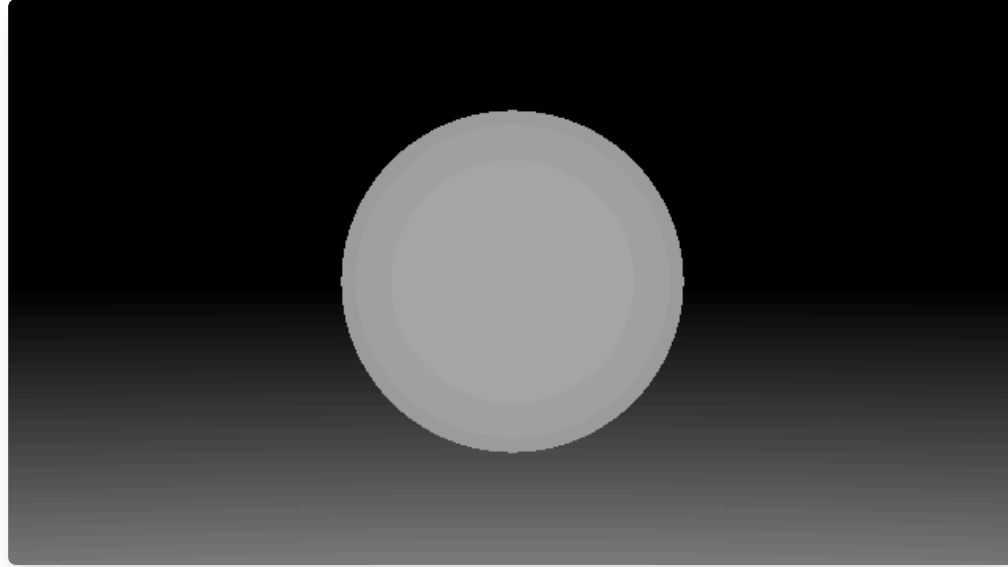
Object ID



Surface Normals



Depth



A2: Ray Casting

Check the course github for links to assignments: <https://github.com/karansher/computer-graphics-csc317>

Quiz 1 will be released tomorrow.